CS 61A
Past Exam Practice — Generators and Counting

Your Name: _____

Your Student ID: _____

Room you are taking the exam in: _____

Name and SID of the person to your **left**: _____

Name and SID of the person to your **right**: _____

You have 110 minutes. There are 8 questions of varying credit. (39 points total)

Question	1	2	3	4	5	6	7	8	Total
Points	0	8	7	6	6	7	0	5	39

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◎ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ■ You can select
- ■ multiple squares
- ☑ (Don't do this)

Make sure to **write your student ID at the top of each page** in the provided blank.

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam. I will not use any other unauthorized materials or devices during this exam. I will not communicate with anyone other than the proctors during this exam. I will not share any information about the exam with anyone until after the end of the exam period.

Acknowledge that you have read and agree to the honor code above and **sign your name below**:

_____

This content is protected and may not be shared, uploaded, or distributed.

Q1 *Iterators Are Inevitable* **(0 points)**

Original source: Spring 2019 Final - Question 1 | **Level:** Regular

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. The first row is completed for you.

- If an error occurs, write **Error**, but include all output displayed before the error.
- To display a function value, write **Function**.
- To display an iterator/generator value, write **Iterator**.
- If an expression would take forever to evaluate, write **Forever**.

The interactive interpreter displays the contents of the `repr` string of the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the code below first, then you evaluate each expression in the table in the order shown. Expressions evaluated by the interpreter have a cumulative effect.

```
def love():
    yield 1000
    yield from [2000, 3000]

x = love()
L = list(x)

def alternate(real, ity):
    i1, i2 = iter(real), iter(ity)
    try:
        while True:
            yield next(i1)
            yield next(i2)
    except StopIteration:
        yield 'inevitable'

thanos = ['power', 'space', 'reality']
tony = ['mind', 'soul', 'time']
i = iter(tony)
next(i)
tony.extend(list(i))
thanos = tony[2::-2]
```

Expression	**Output**
`pow(10, 2)`	`100`
`print(print('end', print('game')), x)`	
`L`	
`next(x)`	
`tony`	
`list(alternate(thanos[1:], thanos))`	

This content is protected and may not be shared, uploaded, or distributed.

Q2 *Hog Revisited* (8 points)

Original source: Fall 2022 Midterm 2 - Question 3 | **Level:** Regular

Represent a die by an iterator over its outcomes. Each call to `next(dice)` produces the next roll.

Part A: Cycling outcomes. Implement `make_test_dice` so that it repeatedly yields the values in `outcomes`, in order, forever. Assume `outcomes` is a non-empty list.

```
def make_test_dice(outcomes):
    """Yield the outcomes repeatedly, in their original order.

    >>> dice = make_test_dice([1, 2, 3])
    >>> [next(dice) for _ in range(8)]
    [1, 2, 3, 1, 2, 3, 1, 2]
    """
    outcomes = list(outcomes)
    while True:
        for i in _______:  # (a)
            _______  # (b)
```

Q2.1 (1 point) Which expressions could fill in blank (a)?

- ◯ `outcomes`
- ◯ `outcomes + make_test_dice(outcomes)`
- ◯ `outcomes.extend(make_test_dice(outcomes))`
- ◯ `outcomes[0] + make_test_dice(outcomes[1:])`
- ◯ `make_test_dice(outcomes) + make_test_dice(outcomes)`
- ◯ `make_test_dice(outcomes[0]) + make_test_dice(outcomes[1:])`

Q2.2 (2 points) Fill in blank (b).

- ◯ `yield i`
- ◯ `yield outcomes`
- ◯ `yield outcomes[i]`
- ◯ `yield from i`
- ◯ `return i`

This content is protected and may not be shared, uploaded, or distributed.

(Question 2 continued…)

Part B: Scoring a turn. Implement `roll_dice`. It must consume exactly `num_rolls` outcomes from the iterator. If any outcome is 1, return 1; otherwise, return their sum. Assume `num_rolls` is positive.

```
def roll_dice(num_rolls, dice):
    """Score a turn while consuming every requested roll.

    >>> dice = make_test_dice([6, 6, 6, 6, 6, 1])
    >>> roll_dice(5, dice)  # 6, 6, 6, 6, 6
    30
    >>> roll_dice(5, dice)  # 1, 6, 6, 6, 6
    1
    >>> roll_dice(2, dice)  # 6, 1
    1
    >>> roll_dice(10, make_test_dice([2, 4, 3]))
    29
    """
    rolls = _______  # (c)
    if _______:  # (d)
        return 1
    _______  # (e)
```

Q2.3 (2 points) Fill in blank (c).

- ◯ `[next(dice) for _ in range(num_rolls)]`
- ◯ `[next(dice) for _ in range(num_rolls - 1)]`
- ◯ `[dice for _ in range(num_rolls)]`
- ◯ `next(dice)`
- ◯ `list(dice)`

Q2.4 (1 point) Fill in blank (d).

- ◯ `1 in rolls`
- ◯ `1 not in rolls`
- ◯ `rolls == 1`
- ◯ `rolls[0] == 1`
- ◯ `sum(rolls) == 1`

Q2.5 (2 points) Fill in blank (e).

- ◯ `return sum(rolls)`
- ◯ `return sum(dice)`
- ◯ `return len(rolls)`
- ◯ `return max(rolls)`
- ◯ `return rolls`

This content is protected and may not be shared, uploaded, or distributed.

Q3 *Do You Yield?* **(7 points)**

Original source: Fall 2019 Midterm 2 - Question 5 (Part B adapted to counting a line's executions) | **Level:** Regular

Implement the generator `partitions(n, m)`. Each yielded string describes one way to write positive integer `n` as a sum of non-increasing positive integers no larger than positive integer `m`. Yield each partition exactly once, in the order shown.

Do not add `lambda`, `if`, `and`, `or`, lists, tuples, or dictionaries beyond those already present in the template. Blanks (c) and (e) each contain the two arguments of a recursive call, separated by a comma.

```
1  def partitions(n, m):
2      """Yield partitions of n with parts no larger than m.
3
4      >>> list(partitions(1, 1))
5      ['1']
6      >>> list(partitions(2, 2))
7      ['2', '1 + 1']
8      >>> list(partitions(4, 2))
9      ['2 + 2', '2 + 1 + 1', '1 + 1 + 1 + 1']
10     """
11     if _______:  # (a)
12         yield _______  # (b)
13     if n > 0 and m > 0:
14         for p in partitions(_______):  # (c)
15             yield _______  # (d)
16         yield from partitions(_______)  # (e)
```

Q3.1 (2 points) Fill in blanks (a) and (b).

(a)

(b)

Q3.2 (1 point) Fill in blank (c).

- ◯ `n - m, m`
- ◯ `n - 1, m`
- ◯ `n, m`
- ◯ `n - m, m - 1`
- ◯ `n, m - 1`

Q3.3 (2 points) Fill in blank (d).

(d)

Q3.4 (1 point) Fill in blank (e).

- ◯ `n, m - 1`
- ◯ `n - m, m - 1`
- ◯ `n, m`
- ◯ `n - m, m`
- ◯ `n - 1, m`

Part B: Counting a line. Assume `partitions` is implemented correctly. For a positive integer `n`, how many times does line X run?

```
1  for p in partitions(n, 2):
2      print(p)  # X
```

Q3.5 (1 point) Select the Python expression in `n` for the number of times line X runs.

- ◯ `n // 2 + 1`
- ◯ `(n + 1) // 2`
- ◯ `n + 1`
- ◯ `n // 2`
- ◯ `n`

This content is protected and may not be shared, uploaded, or distributed.

Q4 *Off to the Races* **(6 points)**

Original source: Spring 2026 Midterm 2 - Question 9 (adapted to a single-line count and loop iteration counts) | **Level:** Regular

Both functions keep only the words shorter than `length`. Each question asks for one count for an input linked list containing `n` words. For loops, count how many times the body runs; do not count condition checks.

Recursive version.

```
1  def remove_long_words1(links, length):
2      if not isinstance(links, Link):
3          return ()
4      if len(links.first) >= length:  # A
5          return remove_long_words1(links.rest, length)
6      return Link(links.first, remove_long_words1(links.rest, length))
```

Q4.1 (2 points) For an input list of `n` words, how many times does line A run in total?

Iterative version. This version can change the links in the input.

```
1  def remove_long_words2(links, length):
2      head = links
3      while isinstance(head, Link) and len(head.first) >= length:  # Loop 1
4          head = head.rest
5      if not isinstance(head, Link):
6          return ()
7      prev = head
8      curr = head.rest
9      while isinstance(curr, Link):  # Loop 2
10         if len(curr.first) >= length:
11             prev.rest = curr.rest
12         else:
13             prev = curr
14         curr = curr.rest
15     return head
```

Q4.2 (2 points) Suppose the input contains `n >= 1` words and the first word is shorter than `length`. How many times does the body of Loop 2 run?

Q4.3 (2 points) Now suppose all `n` words have length greater than or equal to `length`. How many times does the body of Loop 1 run?

This content is protected and may not be shared, uploaded, or distributed.

Q5 *Mapping and Counting* **(6 points)**

Original source: Spring 2022 Midterm 2 - Question 6 (adapted to loop iteration counts and a comparison of the loops) | **Level:** Regular

Each implementation returns a Python list containing `f(x)` for each element of an input linked list. Assume `f` returns normally, does not mutate the input, and does not call either mapping function. Each question concerns one use of an implementation on a linked list containing `n` elements. Count loop body executions, excluding condition checks and work inside `f`.

Version 1: Build the Python list directly.

```
def maplink_to_list1(f, lnk):
    """Apply f to each linked-list element, preserving order.

    >>> maplink_to_list1(lambda x: x * x, Link(3, Link(4, Link(5))))
    [9, 16, 25]
    """
    new_lst = []
    while isinstance(lnk, Link):  # Loop 1
        new_lst.append(f(lnk.first))
        lnk = lnk.rest
    return new_lst
```

Q5.1 (2 points) For an input list containing `n` elements, how many times does the body of Loop 1 run?

Version 2: Build a mapped linked list, then copy its values.

```
def maplink_to_list2(f, lnk):
    """Apply f to each linked-list element, preserving order."""
    def map_link(f, lnk):
        if not isinstance(lnk, Link):
            return ()
        return Link(f(lnk.first), map_link(f, lnk.rest))
    mapped_lnk = map_link(f, lnk)
    new_lst = []
    while isinstance(mapped_lnk, Link):  # Loop 2
        new_lst.append(mapped_lnk.first)
        mapped_lnk = mapped_lnk.rest
    return new_lst
```

Q5.2 (2 points) For an input list containing `n` elements, how many times does the body of Loop 2 run?

Q5.3 (2 points) For inputs of the same length, does either loop run more times than the other, or do they run the same number of times? Compare only the loop bodies.

This content is protected and may not be shared, uploaded, or distributed.

Q6 *Thumper* **(7 points)**

Original source: Fall 2021 - Final - Question 2(c) | **Level:** Challenge

A *worm* is a non-negative integer in which the absolute difference between each pair of adjacent digits is 1. For example, 4345 is a worm, but 4334 and 4354 are not. All non-negative integers below 10 are worms.

Implement `thumper`, a generator function that takes a positive integer `k` and a digit `m`. It yields all `k`-digit worms with digits that are all less than or equal to `m`, and it yields these results in increasing order. Assume the number 0 has 0 digits.

```
def thumper(k, m):
    """Yield all k-digit worms with digits that are at most m, in increasing order.

    >>> list(thumper(1, 7))  # Note: 0 has no digits, so it is not a 1-digit worm.
    [1, 2, 3, 4, 5, 6, 7]
    >>> list(thumper(2, 3))
    [10, 12, 21, 23, 32]
    >>> list(thumper(3, 3))
    [101, 121, 123, 210, 212, 232, 321, 323]
    """
    if k == 1:
        _______  # (a)
    else:
        for w in _______:  # (b)
            if _______:  # (c)
                yield 10 * w + (w % 10 - 1)
            if _______:  # (d)
                yield 10 * w + (w % 10 + 1)
```

Q6.1 (3 points) Fill in blank (a).

(a)

Q6.2 (2 points) Fill in blank (b).

(b)

Q6.3 (1 point) Which of these could fill in blank (c)?

- ◯ `True`
- ◯ `False`
- ◯ `w > 0`
- ◯ `w >= 0`
- ◯ `w < m`
- ◯ `w <= m`
- ◯ `w % 10 > 0`
- ◯ `w % 10 >= 0`
- ◯ `w % 10 < m`
- ◯ `w % 10 <= m`

Q6.4 (1 point) Which of these could fill in blank (d)?

- ◯ `True`
- ◯ `False`
- ◯ `w > 0`
- ◯ `w >= 0`
- ◯ `w < m`
- ◯ `w <= m`
- ◯ `w % 10 > 0`
- ◯ `w % 10 >= 0`
- ◯ `w % 10 < m`
- ◯ `w % 10 <= m`

This content is protected and may not be shared, uploaded, or distributed.

Q7 *We're All the Same to Them* **(0 points)**

Original source: Summer 2017 Final - Question 1 | **Level:** Challenge

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. If an error occurs, write "Error", but include all output displayed before the error. If a function value is displayed, write "Function". If an iterator or generator value would be displayed, write "Iterator" (instead of something like `<iterator object at 0x...>`).

Recall: The interactive interpreter displays the value of a successfully evaluated expression, unless it is `None`.

Assume that you have started `python3` and executed the following statements:

```
def tee(iterable):
    it = iter(iterable)
    queues = [[], []]
    def gen(lst):
        while True:
            if not lst:
                try:
                    value = next(it)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)
            yield lst.pop(0)
    return [gen(queues[0]), gen(queues[1])]

yum = ['avocado', 'quinoa', 'cream cheese']
```

Expressions are evaluated in the order shown and have a cumulative effect. The first row is completed for you.

Expression	**Output**
`print(yum[0] + ' ' + yum[-1]) * 5`	`avocado cream cheese` `Error`
`print(print(next(iter(yum))), next(yum))`	
`next(iter(next(iter(yum))))`	

`eat = iter(tee(yum))`
`neat = next(eat)`
`next(neat)`	

`yum[0] = yum.pop()`
`yum.append(next(neat))`
`yum[:2]`	

`next(neat)`	
`munch = next(iter(tee(yum)))` `next(munch) + ' ' + next(next(eat))`	

This content is protected and may not be shared, uploaded, or distributed.

Q8 *Trictionary or Treat* **(5 points)**

Original source: Fall 2018 Midterm 2 - Question 7 | **Level:** Challenge

A trictionary consists of a key tree `k` and a value tree `v` with the same shape. A key's values are the labels at the corresponding positions in `v`; keys may repeat. Implement `lookups(k, key)` to yield one function per match. Each function accepts a value tree and returns the label at that match's position. Complete `new_lookup` as well.

```
k = Tree(5, [
    Tree(7, [Tree(2)]),
    Tree(8, [Tree(3), Tree(4)]),
    Tree(5, [Tree(4), Tree(2)])])
v = Tree('Go', [
    Tree('C', [Tree('C')]),
    Tree('A', [Tree('S'), Tree(6)]),
    Tree('L', [Tree(1), Tree('A')])])

def lookups(k, key):
    """Yield a lookup for each occurrence of key in k.

    >>> [f(v) for f in lookups(k, 2)]
    ['C', 'A']
    >>> [f(v) for f in lookups(k, 3)]
    ['S']
    >>> [f(v) for f in lookups(k, 6)]
    []
    """
    if _______:  # (a)
        yield lambda v: _______  # (b)
    for i in range(len(k.branches)):
        _______:  # (c)
            yield new_lookup(i, lookup)

def new_lookup(i, f):
    def g(v):
        return _______  # (d)
    return g
```

Q8.1 (2 points) Fill in blanks (a) and (b).

(a)	(b)

Q8.2 (2 points) Fill in blank (c).

- ◯ `for lookup in lookups(k.branches[i], key)`
- ◯ `for lookup in lookups(k.branches[i], key.label)`
- ◯ `for lookup in lookups(k, key)`
- ◯ `for lookup in k.branches[i]`
- ◯ `for lookup in lookups(k.branches, key)`

Q8.3 (1 point) Fill in blank (d).

- ◯ `f(v.branches[i])`
- ◯ `f(v.branches)`
- ◯ `v.branches[i]`
- ◯ `f(v)`
- ◯ `f(v.branches[i].label)`

This content is protected and may not be shared, uploaded, or distributed.